

Assignment 2 - Data Ingestion & Data Warehouse Modelling

Onboarding Flow Optimization A/B Test

Aishwarya Mishra

August 2026

Deliverables vs Rubric

1. How to Run
2. Layer Design
 - 2.1 Raw
 - 2.2 Staging
 - 2.3 Warehouse (star schema)
 - 2.4 Data Mart
3. Data Quality
4. Sample Output

Deliverables vs Rubric

Rubric item	Where it's covered
Database Design (2 marks)	Section 2 (layer design) + schema/
Data Ingestion (1 mark)	build_db.py load_raw_events()
SQL Transformations (1 mark)	schema/02_staging.sql , schema/03_warehouse.sql , schema/04_datamart.sql
Data Quality (1 mark)	schema/05_data_quality_checks.sql + dirty-row injection in build_db.py

1. How to Run

```
in pip install -r ../requirements.txt # stdlib only, sqlite3 is built
python build_db.py \
  --events-csv ../assignment1/generated_data/events.csv \
  --stepmap-json ../assignment1/stepmap.json \
  --db db/ab_test.db
```

This ingests the Assignment 1 CSV, injects a handful of deliberately malformed rows (null keys, invalid enums, a duplicate delivery) to prove the validation logic actually catches something, then builds all four layers in order and runs the data quality checks, printing a pass/fail report and a KPI preview.

2. Layer Design

```
raw_events                                (RAW)
  | validate + dedupe + json_extract flatten
  v
stg_events, stg_events_rejects            (STAGING)
  | dimensional modelling
  v
dim_user, dim_date, dim_step,
fact_onboarding_step_events,
fact_user_funnel                          (WAREHOUSE)
  | group-by aggregation
  v
mart_kpi_by_group,
mart_guardrails_by_group,
mart_step_dropoff,
mart_daily_funnel                         (DATA MART)
```

2.1 Raw

`raw_events` mirrors the source CSV column-for-column, no casting, no filtering. Bad data is expected to land here - that's what staging is for.

2.2 Staging

`stg_events` de-duplicates on `event_id` (keep-first), rejects rows with null keys or out-of-domain `test_group/platform` values into `stg_events_rejects` (rather than silently dropping them), casts `event_timestamp` to a real datetime, and flattens the JSON properties blob into typed columns via SQLite's `json_extract` - so no downstream SQL needs to touch raw JSON.

2.3 Warehouse (star schema)

- **dim_user** - one row per user: `test_group`, `platform`, `install date`, `account-creation flag/method`. Grain: 1 row per `user_id`.
- **dim_date** - a generated calendar spine over the observed date range, for time-series joins.
- **dim_step** - the step -> variant -> bucket mapping from `stepmap.json` (design metadata, loaded directly rather than derived from events).
- **fact_onboarding_step_events** - granular grain: 1 row per step-viewed/completed/abandoned event. Used for step-level drop-off analysis.
- **fact_user_funnel** - the primary analytical fact, 1 row per user, with every funnel/guardrail flag and timestamp needed to compute every KPI in the event tracking plan via a single GROUP BY `test_group`.

2.4 Data Mart

Pre-aggregated, dashboard-ready tables, built as tables (not views) so Assignment 4's dashboard queries are cheap:

- `mart_kpi_by_group` - completion, activation, paid conversion, time-to-first-lesson, D1/D7/D30 return, one row per test group.
- `mart_guardrails_by_group` - relevance score, refund/cancellation rate, support ticket rate, notification opt-in, one row per test group.
- `mart_step_dropoff` - per-step abandon rate per group, for funnel charts.
- `mart_daily_funnel` - completion rate by install date x group, for trend charts.

3. Data Quality

`schema/05_data_quality_checks.sql` runs 12 checks after the pipeline builds: row counts, reject counts, duplicate detection, referential integrity (`dim_user <-> fact_user_funnel`), funnel-order sanity (nobody activated without completing onboarding, nobody purchased without activating), enum validity, and rate bounds (`[0, 1]`). `build_db.py` prints every check's result; a healthy run shows 0 for everything except `raw_row_count` and `staging_rejects` (which is intentionally non-zero - see Section 1).

4. Sample Output

From a run against the 6,000-user generated dataset (see [../assignment1/README.md](#) for how it was made):

test_group	users_started	onboarding_completion_rate	activation_rate
A	2000	0.574	0.698
B	2000	0.730	0.632
C	2000	0.815	0.656

Full pipeline context: see the [repo root README](#).